

MINIMUM SPANNING TREE VS SHORTEST PATH

Greedy Algorithm Comparison

Design and Analysis of Algorithms (CSA06) Course Outcome: C04 Assessment Tool 3 – Q39 Prepared as an 8-

page implementation-based report

Abstract

This report presents an implementation-based comparison of Minimum Spanning Tree (MST) and Shortest Path graph optimization. Q39 requires a graph to be used to compute an MST and a shortest path, compare objectives and outputs, analyze greedy selection strategies, evaluate complexity, discuss applications, and generate graph-optimization insights. The supplied implementation uses Kruskal's algorithm for the MST and Dijkstra's algorithm for the shortest path.

Because Q39 does not specify a particular graph, the implementation uses a representative undirected weighted graph with five vertices and eight edges. The computed MST contains four edges with total cost 4. Dijkstra's algorithm, using source A and destination E, obtains the direct path $A \rightarrow E$ with cost 3. Graphical outputs are included to show the original graph, greedy selection, final tree, shortest path, and cost comparison.

Keywords

Minimum Spanning Tree, Kruskal, Shortest Path, Dijkstra, Greedy Algorithm, Weighted Graph, Graph Optimization.

Page

1. Introduction and Problem Definition

A weighted graph is a collection of vertices connected by edges, where each edge has a numerical weight such as distance, cost, time, or bandwidth. Graph optimization algorithms are used when the objective is to select an efficient structure or route from many possible alternatives.

The two problems considered in this report are related but fundamentally different. A Minimum Spanning Tree connects every vertex while minimizing the sum of selected edge weights. A Shortest Path problem finds a minimum-cost route between a selected source and destination.

1.1 Q39 Requirements

Compute a Minimum Spanning Tree and a shortest path for a graph.

Compare their objectives and outputs.

Analyze their greedy selection strategies.

Evaluate computational complexity.

Discuss practical applications.

Generate insights about graph optimization and algorithm selection.

1.2 Selected Experimental Parameters

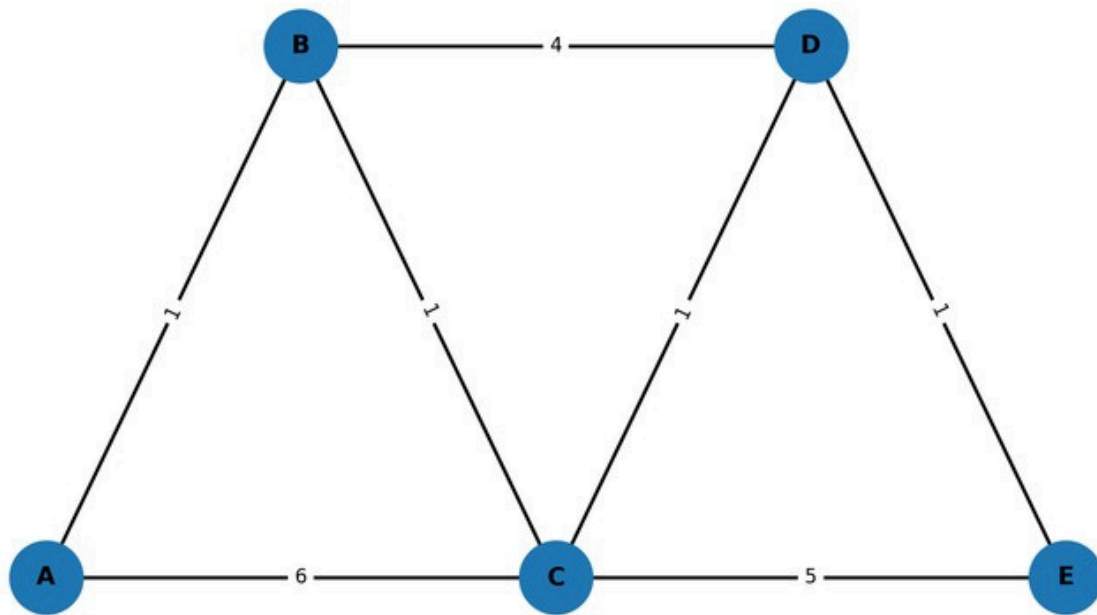
Parameter	Value
Graph type	Undirected weighted graph
Vertices	A, B, C, D, E
Number of vertices (V)	5
Number of edges (E)	8
MST algorithm	Kruskal's Algorithm
Shortest-path algorithm	Dijkstra's Algorithm
Source	A
Destination	E

1.3 Why These Parameters Were Selected

The question statement does not supply specific vertices, edges, or weights. Therefore, a representative graph is explicitly selected for demonstration. The graph contains enough alternative edges to make

both greedy decisions visible and to show that an MST and a shortest path can have different objectives and results.

Figure 1. Original Weighted Graph



CODE

```
# =====
# Q39. MINIMUM SPANNING TREE VS SHORTEST PATH
# =====

# =====
# 1. IMPORT REQUIRED LIBRARIES
# =====

import networkx as nx
import matplotlib.pyplot as plt
import heapq

# =====
# 2. DEFINE THE GRAPH
# =====

vertices = ['A', 'B', 'C', 'D', 'E']

edges = [
    ('A', 'B', 1),
    ('B', 'C', 1),
    ('C', 'D', 1),
    ('D', 'E', 1),
    ('A', 'E', 3),
    ('B', 'D', 4),
    ('C', 'E', 5),
    ('A', 'C', 6)
]

source = 'A'
destination = 'E'
```

```
# =====
# FINAL RESULT
# =====

print("\n")
print("=" * 90)
print("FINAL RESULT")
print("=" * 90)

print("\nMinimum Spanning Tree:")

for u, v, w in mst_edges:
    print(
        f"{u} - {v} = {w}"
    )

print(
    f"\nTotal MST Cost = {mst_cost}"
)

print(
    "\nShortest Path:"
)

print(
    " -> ".join(shortest_path)
)

print(
    f"Shortest Path Cost = {shortest_path_cost}"
)
```

2. Graph Representation and Minimum Spanning Tree

2.1 Input Graph

Edge	Weight
A – B	1
B – C	1
C – D	1
D – E	1
A – E	3
B – D	4
C – E	5
A – C	6

The graph is represented in the implementation using a vertex list and an edge list. NetworkX is then used to create the graph for visualization. The edge weights are preserved exactly in the implementation.

2.2 Minimum Spanning Tree

An MST is a spanning tree that connects all vertices using the minimum possible total edge weight. A spanning tree of five vertices must contain exactly $V - 1 = 4$ edges.

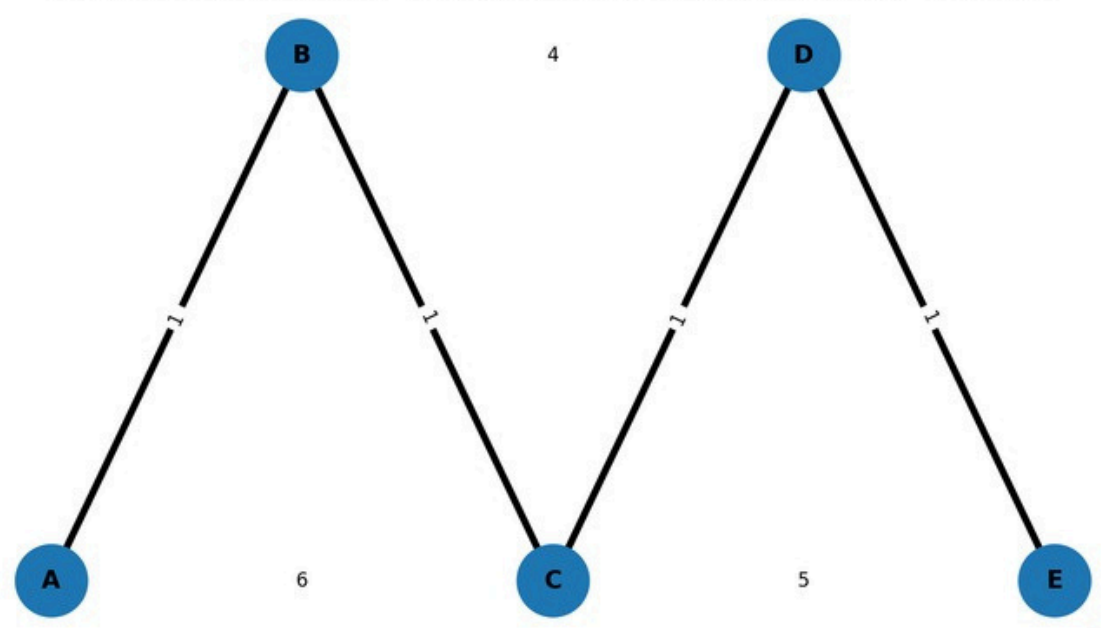
2.3 Kruskal's Greedy Process

Order	Edge	Weight	Action
1	A-B	1	Selected
2	B-C	1	Selected
3	C-D	1	Selected

4	D-E	1	Selected
5	A-E	3	Not needed after V-1 edges

Kruskal's algorithm sorts edges in non-decreasing order and selects the smallest edge that does not create a cycle. The uploaded implementation uses a Disjoint Set (Union-Find) structure with path compression and union by rank for cycle detection.

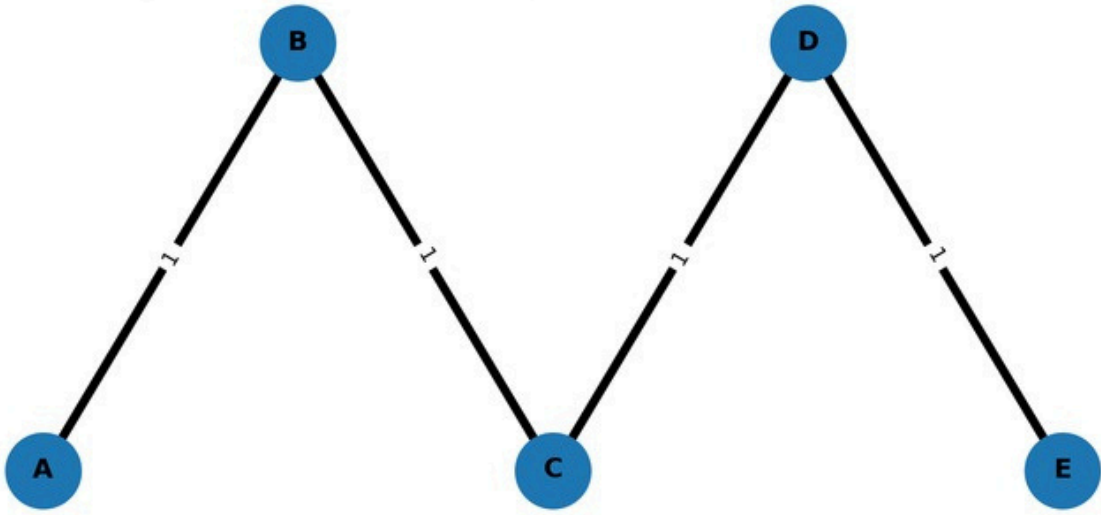
Figure 2. Kruskal Greedy Selection: Solid = Selected, Dashed = Rejected



2.4 Final MST Result

Selected edges are A-B, B-C, C-D, and D-E. Therefore, the total MST cost is $1 + 1 + 1 + 1 = 4$.

Figure 3. Final Minimum Spanning Tree (Total Cost = 4)



3. Shortest Path Using Dijkstra's Algorithm

3.1 Objective

The shortest-path experiment uses A as the source and E as the destination. Dijkstra's algorithm is appropriate for the non-negative edge weights used in this graph.

3.2 Algorithm Procedure

Initialize the source distance to 0 and all other distances to infinity.

Select the unvisited vertex with the smallest tentative distance.

Relax the selected vertex's neighboring edges.

Update a neighbor when a shorter tentative distance is found.

Repeat until the required distances are determined.

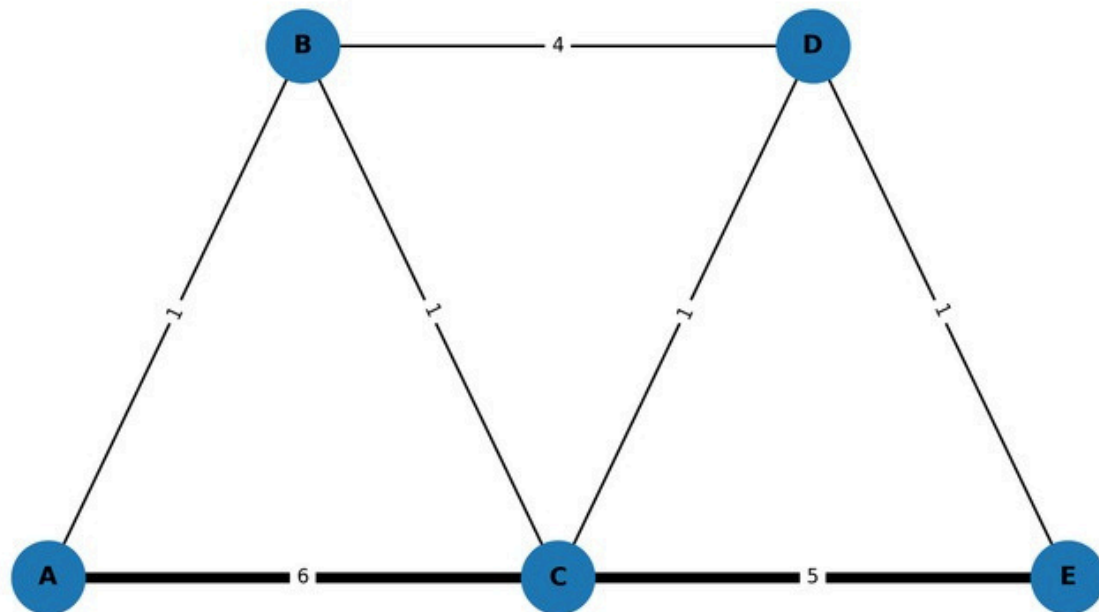
Use predecessor information to reconstruct the final path.

3.3 Main Distance Updates

Stage	Important result
Initialization	A = 0; B = 1; C = 6; D = ∞; E = 3
After B	C becomes 2 and D becomes 5
After C	D becomes 3
Destination E	Direct A–E remains 3

The implementation reconstructs the path using the predecessor dictionary. The resulting path is A → E. Its cost is 3, which is smaller than the chain A → B → C → D → E, whose cost is 4.

Figure 4. Dijkstra Shortest Path: A → E (Cost = 3)



3.4 Final Result

Shortest path: A → E. Shortest-path cost: 3.

The direct A–E edge is not required in the final MST, but it is the shortest route from A to E. This is the central experimental observation: an edge can be useful for a shortest-path objective without being necessary for minimum-cost connectivity.

4. Implementation and Experimental Output

The implementation supplied with this report is the uploaded Google Colab notebook. It defines the graph, constructs an adjacency representation, implements Union-Find for Kruskal's algorithm, computes the MST, runs Dijkstra's algorithm, reconstructs the shortest path, and generates graphical outputs using NetworkX and Matplotlib.

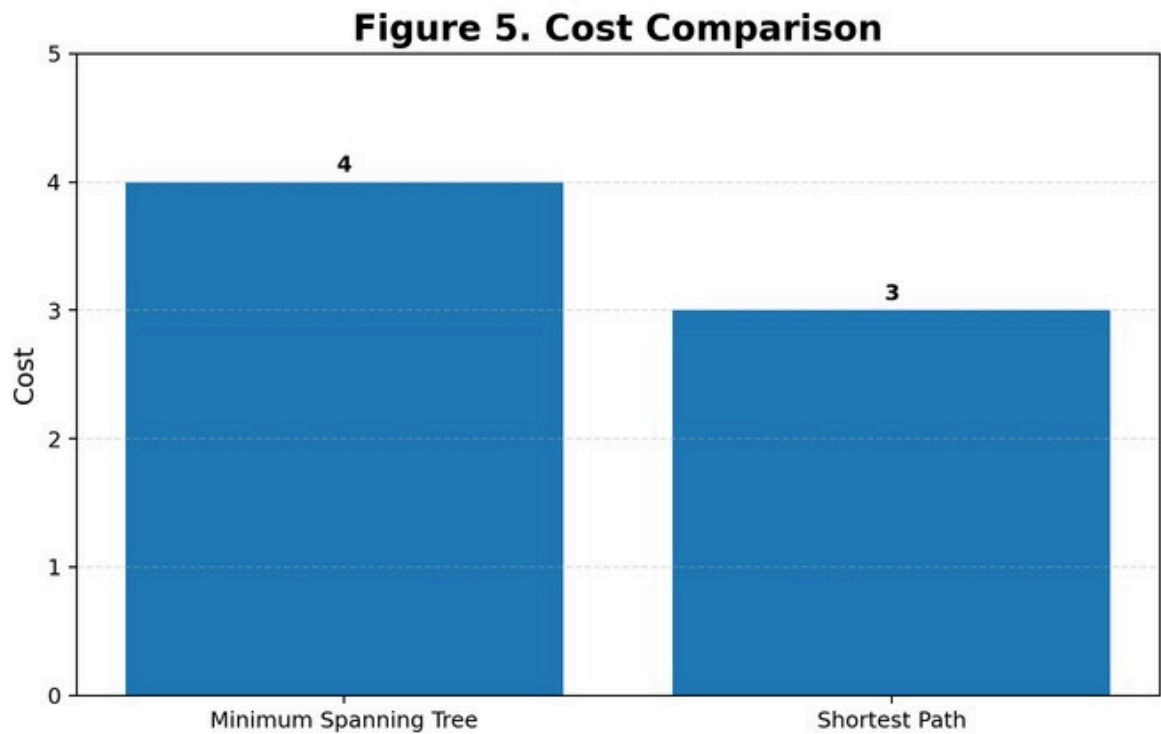
4.1 Implementation Flow

Input weighted graph → Create NetworkX graph → Display original graph → Sort edges → Kruskal/Union-Find → Final MST → Build adjacency list → Dijkstra → Reconstruct shortest path → Generate comparison graphics → Print analysis.

4.2 Core Results

Algorithm	Output	Cost
Kruskal – MST	A-B, B-C, C-D, D-E	4
Dijkstra – Shortest Path	A → E	3

4.3 Visual Cost Comparison



4.4 Implementation Consistency

The report uses the same graph, source, destination, MST result, and shortest-path result as the uploaded implementation. The Dijkstra code uses a simple scan of the vertex list to select the minimum tentative distance. Therefore, its implementation-specific time complexity is $O(V^2 + E)$, not the binary-

heap complexity $O((V + E) \log V)$.

Python 3.x

Google Colab or Jupyter Notebook

NetworkX

Matplotlib

OUTPUT

```
=====
KRUSKAL'S ALGORITHM
=====

Edges sorted by weight:

A - B = 1
B - C = 1
C - D = 1
D - E = 1
A - E = 3
B - D = 4
C - E = 5
A - C = 6

Greedy Selection Process:

SELECTED : A - B = 1
SELECTED : B - C = 1
SELECTED : C - D = 1
SELECTED : D - E = 1

-----
FINAL MST EDGES:
A - B = 1
B - C = 1
C - D = 1
D - E = 1

Total MST Cost = 4
```

```
=====
DIJKSTRA'S ALGORITHM
=====

Source      : A
Destination : E

Greedy Selection Process:

Selected vertex: A (distance = 0)
  Updated B: 1
  Updated E: 3
  Updated C: 6
Selected vertex: B (distance = 1)
  Updated C: 2
  Updated D: 5
Selected vertex: C (distance = 2)
  Updated D: 3
Selected vertex: D (distance = 3)
Selected vertex: E (distance = 3)

-----
Shortest Path:
A -> E

Shortest Path Cost = 3

-----
FINAL SHORTEST DISTANCES FROM SOURCE A
=====
Vertex      Distance from A
-----
A            0
B            1
C            2
D            3
E            3
```

```
=====
FINAL RESULT
=====

Minimum Spanning Tree:
A - B = 1
B - C = 1
C - D = 1
D - E = 1

Total MST Cost = 4

Shortest Path:
A -> E

Shortest Path Cost = 3
```

MST VS SHORTEST PATH - COMPARATIVE ANALYSIS		
Parameter	Minimum Spanning Tree	Shortest Path
Objective	Connect all vertices	Minimum route between vertices
Algorithm	Kruskal	Dijkstra
Greedy Selection	Smallest safe edge	Nearest unvisited vertex
Output	Tree	Path
Number of Edges	4	1
Cost	4	3
Complexity	$O(E \log E)$	$O(V^2 + E)$
Applications	Network design	Navigation / routing

```
=====
CONCLUSION:
MST minimizes the total cost of connecting all vertices.
Shortest Path minimizes the cost between a selected source and destination.
Therefore, MST and Shortest Path are different graph optimization problems even though both can use greedy strategies.
=====
```

NOTE : All graphs and trees are already used in the report as the figures

5. Comparative Analysis

Criterion	Minimum Spanning Tree	Shortest Path
Primary objective	Minimize total cost of connecting all vertices	Minimize route cost between selected vertices
Algorithm used	Kruskal	Dijkstra
Greedy decision	Smallest safe edge	Closest unvisited vertex
Output	Tree	Path
Source required	No	Yes
Destination required	No	Yes for a specific route
Cycle handling	Explicitly avoids cycles	A shortest route is represented without cycles
Experimental result	A-B, B-C, C-D, D-E	A → E
Experimental cost	4	3
Implementation complexity	$O(E \log E)$	$O(V^2 + E)$
Typical application	Network construction	Navigation/routing

5.1 Greedy Strategy Differentiation

Kruskal makes a global edge-based decision. It considers the next smallest edge and checks whether the edge is safe with respect to cycle formation. Dijkstra makes a source-based distance decision. It finalizes the currently closest unvisited vertex and uses that decision to relax neighboring distances.

5.2 Why the Outputs Differ

The MST must connect all five vertices, so it needs four edges. The shortest-path experiment only needs a route from A to E. The direct edge A-E has weight 3 and is therefore the best A-to-E route, while the four weight-1 edges are collectively better for connecting the entire graph.

5.3 Complexity

Algorithm	Implementation complexity	Space
-----------	---------------------------	-------

Kruskal	$O(E \log E)$	$O(V + E)$
Dijkstra (this implementation)	$O(V^2 + E)$	$O(V + E)$
Dijkstra with binary heap (reference)	$O((V + E) \log V)$	$O(V + E)$

6. Critical Evaluation, Applications and Insights

6.1 Advantages and Limitations

Method	Advantages	Limitations
MST / Kruskal	Minimizes total network connection cost; avoids cycles; effective for sparse graphs.	Does not guarantee the shortest route between arbitrary vertices; requires connectivity for one spanning tree.
Shortest Path / Dijkstra	Finds minimum-cost routes from a source; useful for navigation and routing; efficient for non-negative weights.	Dijkstra is not suitable for negative edge weights; it does not minimize total network connection cost.

6.2 Real-World Applications

Minimum Spanning Tree

- Computer and communication network design
- Electrical distribution and infrastructure planning
- Telecommunication connectivity
- Minimum-cost cable, pipeline, or link planning

Shortest Path

- GPS and navigation
- Bus and transportation route planning
- Internet and packet routing
- Robot path planning

6.3 Synthesis and Insights

- MST minimizes the total cost of connecting all vertices, not the travel cost between one pair.
- Shortest Path minimizes a route from a selected source to a destination; it does not minimize the cost of the whole network.
- The same graph can produce different selected edges because the optimization objectives are different.
- Greedy choice is problem-specific: Kruskal selects safe minimum-weight edges, while Dijkstra selects the closest unvisited vertex.

Algorithm selection should be based on the objective rather than simply on the fact that both algorithms are greedy.

6.4 Algorithm Selection Guide

Requirement	Recommended approach
Minimum-cost connection of all locations	Kruskal or Prim

Shortest route from one source with non-negative weights	Dijkstra
Shortest paths with negative edge weights	Bellman-Ford
All-pairs shortest paths	Floyd-Warshall

7. Conclusion, Future Scope and References

7.1 Final Results

Result	Value
Vertices	5
Edges	8
MST edges	A-B, B-C, C-D, D-E
MST total cost	4
Shortest-path source	A
Shortest-path destination	E
Shortest path	A → E
Shortest-path cost	3

7.2 Conclusion

The implementation successfully demonstrates the difference between two important greedy graph optimization problems. Kruskal's algorithm constructs a Minimum Spanning Tree with total cost 4, connecting all five vertices using four selected edges. Dijkstra's algorithm finds the shortest path from A to E with cost 3.

The results should not be interpreted as one algorithm being better than the other. The algorithms solve different objectives. Kruskal is appropriate when the requirement is minimum-cost network connectivity, whereas Dijkstra is appropriate when the requirement is minimum-cost routing from a source. The graphical outputs make the difference between the complete graph, the selected MST, and the shortest route directly observable.

7.3 Future Scope

Allow users to enter vertices and weighted edges dynamically.

Read larger graphs from CSV or other data files.

Add Prim's algorithm for a second MST comparison.

Add Bellman-Ford for negative-weight shortest paths.

Use a priority queue to optimize Dijkstra for larger sparse graphs.

Develop an interactive graph visualization for route and network analysis.

7.4 References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C., Introduction to Algorithms, MIT Press.

Kleinberg, J. and Tardos, É., Algorithm Design, Pearson.

Dasgupta, S., Papadimitriou, C., and Vazirani, U., Algorithms, McGraw-Hill.

SIMATS Engineering, CSA06 – Design and Analysis of Algorithms, Assessment Tool 3 Question Bank & Rubrics, Q39.